

Arbitrary Precision Arithmetic

Final Project Report
CS1023 - Software Development Fundamentals

Anant Maheshwary
CS24BTECH11006

April 2025

Contents

1	Abstract	2
2	Java Classes	2
2.1	AInteger	2
2.1.1	Data Representation	2
2.1.2	Constructors	2
2.1.3	Methods	3
2.2	AFloat	3
2.2.1	Data Representation	3
2.2.2	Constructors	4
2.2.3	Methods	5
3	File Structure and Implementation Techniques	6
3.1	Project Structure	6
3.2	Implementation Techniques	6
3.3	Example Usage	7
4	Testing	7
5	Containerization with Docker	8
5.1	Build the Docker Image	8
5.2	Run Java CLI in a Container	8
5.3	Run Python Test Script in a Container	8
5.3.1	Randomized Testing	8
5.3.2	Specific Test	9
5.4	CLI Help	9
5.5	Other Helpful Docker Commands	9

1 Abstract

Java's primitive data types, such as `int`, `long`, `float`, and `double`, have limitations in the range and precision of numbers they can represent. For example, a Java `int` can represent values only in the range from $-2,147,483,648$ to $2,147,483,647$, and a `float` has a precision of approximately 7 decimal digits with a range of about $1.4\text{E}-45$ to $3.4\text{E}+38$. These constraints make them unsuitable for computations involving very large integers or highly precise decimal values.

This project addresses these limitations by developing a Java library, `arbitraryarithmetic`, that provides classes for arbitrary-precision integer and floating-point arithmetic. The library includes `AInteger` and `AFloat` classes, implemented using object-oriented principles, to store numerical values of any desired length and precision. These classes support constructors for initialization and methods for addition, subtraction, multiplication, and division, for accurate computations without overflow, underflow or loss of precision.

2 Java Classes

2.1 `AInteger`

2.1.1 Data Representation

It has the following member variables for its representation:

1. `int sign` - To store the sign of the `AInteger`. It takes the values 1 for positive, -1 for negative and 0 for a zero `AInteger`.
2. `int[] num` - An `AInteger` array whose elements are integers in the range 0 to $10^9 - 1$. The `AInteger` to be stored is broken into blocks of 9 digits where the most significant block can have less than 9 digits. The individual integers so obtained are stored in an array in the decreasing order of their significance in the original number. i.e its last 9 digits will form the first element of the array.

E.g the number 1023003456789123456789 is stored as $\{123456789, 3456789, 1023\}$

My Reasoning behind this: `int` value in Java takes the maximum value $= 2^{31} - 1 = 2,14,74,83,647$. So every 9 digit decimal integer can be stored in the `int` variable. Also, sum of two such 9-digit integers still lies within the range of `int`, reducing risk of overflow problems later on.

The `AInteger` representing zero is represented by `sign = 0` and `num` array = $\{0\}$

2.1.2 Constructors

Public Constructors	Description
<code>AInteger()</code>	Default constructor. Initializes the <code>AInteger</code> to zero.
<code>AInteger(String s)</code>	Initializes the <code>AInteger</code> from its string representation <code>s</code> .
<code>AInteger(AInteger obj)</code>	Copy constructor. Initializes a new <code>AInteger</code> object with the same sign and magnitude as <code>obj</code> .

2.1.3 Methods

Public Methods	Description
<code>parse(String s)</code>	A static method that parses the string <code>s</code> and returns an <code>AInteger</code> object. Throws <code>NumberFormatException</code> if string is invalid.
<code>getSign()</code>	Returns the sign of the <code>AInteger</code> (-1 for negative, 0 for zero, 1 for positive).
<code>getNum()</code>	Returns the array representing the magnitude of the <code>AInteger</code> .
<code>toString()</code>	Returns the string representation of the <code>AInteger</code> object.
<code>equals(Object obj)</code>	Returns <code>true</code> if the current <code>AInteger</code> is equal to the object <code>obj</code> , <code>false</code> otherwise.
<code>compare(AInteger integer2)</code>	Compares the current <code>AInteger</code> with <code>integer2</code> . Returns 1 if greater, 0 if equal, -1 if less than.
<code>compareMagnitude(AInteger integer2)</code>	Compares the magnitudes of the current <code>AInteger</code> and <code>integer2</code> .
<code>negate()</code>	Returns an <code>AInteger</code> with the opposite sign.
<code>abs()</code>	Returns the absolute value of the current <code>AInteger</code> .
<code>add(AInteger integer2)</code>	Adds the current <code>AInteger</code> to <code>integer2</code> and returns the result.
<code>subtract(AInteger integer2)</code>	Subtracts <code>integer2</code> from the current <code>AInteger</code> and returns the result.
<code>multiply(AInteger integer2)</code>	Multiplies the current <code>AInteger</code> with <code>integer2</code> and returns the result.
<code>divide(AInteger divisor)</code>	Divides the current <code>AInteger</code> by <code>divisor</code> and returns the result. Throws <code>ArithmeticException</code> if dividing by zero.

2.2 AFloat

2.2.1 Data Representation

It has the following member variables for its representation:

1. `AInteger num` - This `AInteger` stores all the digits of the floating-point number, effectively treating it as an integer with the decimal point removed. The sign of the `AFloat` is also stored within this `AInteger`.
2. `int exp` - It stores the exponent when the above float is written in scientific notation. It can be positive, negative or zero.

E.g. 1234.56789 is stored as `AInteger(123456789)` and `exp=3` as the given number in scientific notation is $1.23456789 * 10^3$.

Similarly 0.00012345 is stored as `AInteger(12345)` and `exp=-4` as the given number in scientific notation is $1.2345 * 10^{-4}$.

The `AFloat 0.0` is stored as `AInteger(0)` and `exp=0`

Also, 1234.56789 will never be stored as `AInteger(1234567890)` and `exp=3`

My Reasoning behind this: To ensure a unique representation for every `AFloat` and to simplify arithmetic operations, the `AInteger num` will never have trailing zeros. Representing the number in scientific notation removes any ambiguity about the location of the decimal point and storing the exponent separately simplifies multiplication and division, as the exponents can be handled independently.

This design of the `AFloat` class demonstrates code reusability by employing the `AInteger` class (as composition) and using its arithmetic methods.

Precision - The `AFloat` class also contains a `static final` variable `PRECISION` which is currently set to 100 and defines the number of digits of precision to which the result of a division is calculated. It can be changed according to requirement. Other operations like addition, multiplication and subtraction give the most accurate result (without truncating) and are not affected by the `PRECISION` variable

The member variables in both classes are declared `final` and `private` to make the classes **immutable** and all operations create new objects instead of modifying earlier ones.

2.2.2 Constructors

Public Constructors	Description
<code>AFloat()</code>	Default constructor. Initializes the <code>AFloat</code> to zero (0.0).
<code>AFloat(AFloat float2)</code>	Copy constructor. Initializes the <code>AFloat</code> as a copy of <code>float2</code> .
<code>AFloat(String s)</code>	Initializes the <code>AFloat</code> from its string representation <code>s</code> .

2.2.3 Methods

Public Methods	Description
<code>parse(String s)</code>	A static method that parses the string <code>s</code> and returns an <code>AFloat</code> object. Throws <code>NumberFormatException</code> if string is invalid.
<code>getNum()</code>	Returns the <code>AInteger</code> representing the numerical value of the <code>AFloat</code> .
<code>getExp()</code>	Returns the exponent of the <code>AFloat</code> .
<code>toString()</code>	Returns the string representation of the <code>AFloat</code> .
<code>add(AFloat float2)</code>	Adds the current <code>AFloat</code> to <code>float2</code> and returns the result.
<code>subtract(AFloat float2)</code>	Subtracts <code>float2</code> from the current <code>AFloat</code> and returns the result.
<code>multiply(AFloat float2)</code>	Multiplies the current <code>AFloat</code> by <code>float2</code> and returns the result.
<code>divide(AFloat float2)</code>	Divides the current <code>AFloat</code> by <code>float2</code> and returns the result. Throws <code>ArithmeticException</code> if dividing by zero.

3 File Structure and Implementation Techniques

3.1 Project Structure

The project follows a standard Maven directory structure:

```
Project1/
|--- MyInfArith/
|   --- MyInfArith.java
|--- src/
|   --- main/
|       --- java/
|           --- arbitraryarithmetic/
|               --- AFloat.java
|               --- AInteger.java
|--- target/
|   --- classes/
|       --- arbitraryarithmetic/
|           --- AFloat.class
|           --- AInteger.class
|   --- ... (other Maven build files)
|--- aarithmetic.jar
|--- pom.xml
|--- test.py
|--- Dockerfile
|--- README.md
|--- report.pdf
```

- `src/main/java/arbitraryarithmetic/`: This directory contains the source code for the arbitrary-precision arithmetic library: `AIInteger.java` and `AFloat.java`
- `target/classes/arbitraryarithmetic/`: This directory contains the compiled `.class` files for the library.
- `aarithmetic.jar`: This is the Java Archive (JAR) file created by Maven. It packages the compiled `AIInteger.class` and `AFloat.class` files. It does not contain a class with a main method and is meant to be used as a library and not an executable jar.
- `pom.xml`: This is the Maven Project Object Model file. It configures the project's build process, including dependencies, plugins, and output settings. Maven uses this file to compile the Java source files and package them into the `aarithmetic.jar` file.
- `MyInfArith.java`: This file is in the project's root directory (in a `MyInfArith` package). It contains a `main` method, and demonstrates the use of `AIInteger` and `AFloat` classes from the `aarithmetic.jar` library.

3.2 Implementation Techniques

- **Library as a JAR:** `aarithmetic.jar` holds the arbitrary-precision arithmetic code, making it a reusable library. Other java programs can use it by including the JAR in their classpath.

- **Command-Line Interface (CLI) via MyInfArith.java:** The MyInfArith class provides a command-line interface to interact with the library. It takes four arguments:

1. The data type (int or float).
2. The arithmetic operation (add, sub, mul, or div).
3. The first operand.
4. The second operand.

The `main` method in `MyInfArith.java` conditionally determines at run-time whether to perform integer or floating-point arithmetic based on the first command-line argument. This is called **dynamic dispatch**.

It also uses the classes' `parse(String s)` methods provided by the `AInteger` and `AFloat` classes to create `AInteger` and `AFloat` objects from their string representations and handles potential `NumberFormatException` if the input is invalid or `ArithmeticException` in case of Division by Zero.

3.3 Example Usage

1. Building the project using Maven

```
$ mvn clean install
```

- `mvn clean`: This command removes any previously built artifacts from the `target` directory.
- `mvn install`: This command compiles the Java source files and packages the compiled classes into the `aarithmetic.jar` file.

2. Providing aarithmetic.jar as class path and compiling MyInfArith

```
$ javac -cp ../home/anant/Programs/SDF/Project1/target/aarithmetic.jar  
MyInfArith/MyInfArith.java
```

3. Providing aarithmetic.jar as class path and executing the compiled MyInfArith.class file

```
$ java -cp ../home/anant/Programs/SDF/Project1/target/aarithmetic.jar:target/  
classes MyInfArith.MyInfArith int add 4 5  
Output: 9
```

4 Testing

The project includes `test.py` for automated testing. It supports:

1. **Specific Testing:** Executes a single operation:

```
/usr/bin/python3 test.py <int/float> <add/sub/mul/div> <operand1>  
<operand2>
```

2. **Randomized Testing:** Generates and executes random test cases:

```
/usr/bin/python3 test.py
```

For each test:

1. It compiles Java code:

```
javac -cp ../home/anant/Programs/SDF/Project1/target/aarithmetric.jar
MyInfArith/MyInfArith.java
```

2. It executes the Java code and captures output:

```
java -cp
../home/anant/Programs/SDF/Project1/target/aarithmetric.jar:target/classes
MyInfArith.MyInfArith <data_type> <operator> <operand1> <operand2>
```

3. Calculates the result of the operation using Python (uses Decimal module for float)
4. Compares the both results (considers 30 digits after the decimal in case of float results) and then prints correct/incorrect.

All test cases are randomized over the following:

- Data types: int, float
- Operators: add, sub, mul, div
- Operands: Large random numbers, with random signs and decimal points (for floats).

This testing ensures the library's correctness.

5 Containerization with Docker

5.1 Build the Docker Image

```
docker build -t <image_name> .
```

The project uses a multi-stage Docker build:

- **Stage 1:** Uses a Maven image to build the JAR.
Base Image: maven:3.9.6-eclipse-temurin-21
- **Stage 2:** Uses a JDK 21 image with Python for running tests.
Base Image: eclipse-temurin:21-jdk-jammy

5.2 Run Java CLI in a Container

```
docker run <image_name> int add 123 456
Output: 579
```

5.3 Run Python Test Script in a Container

5.3.1 Randomized Testing

```
docker run -it --entrypoint python3 <image_name> /app/test.py
```


5.3.2 Specific Test

```
docker run --entrypoint python3 <image_name> /app/test.py float div 10.5 2.5
```

5.4 CLI Help

If no arguments are passed, the container displays usage instructions:

```
docker run <image_name>
```

5.5 Other Helpful Docker Commands

Display all images and their information

```
docker images
```

Display all containers (active and closed)

```
docker ps -a
```

Run the container to start an interactive shell (by overriding the entrypoint)

```
docker run -it --entrypoint "/bin/bash" my-arithmetic-app
```

Remove all closed containers

```
docker container prune
```

Remove an image with its image_name

```
docker rmi <image_name>
```